

OpenClaw

~/.openclaw/openclaw.json

配置实践手册

字段地图 · 运行时语义 · Context Engineering · Tool Policy · Sandbox · MCP · Multi-Agent ·
Gateway · 安全加固

研究基线：OpenClaw 官方文档与实时 Schema 机制

版本标记：2026-08-20

核心目标：不仅知道“字段是什么”，更理解“为什么这样设计，以及如何组合成生产级 Agent Runtime”。

文档说明与阅读原则

最重要的版本原则 OpenClaw 配置采用 JSON5，且插件、Channel 会动态扩展 Schema。网页文档只能作为基线；你当前安装版本的 ``openclaw config schema`` 才是字段、类型、枚举、默认值与 UI 元数据的最终权威。

本手册以 OpenClaw 官方 Configuration、Configuration Reference、Configuration - Agents、Configuration - Tools and Custom Providers、Configuration - Channels、Code Mode、Models、MCP 与 Plugin 文档为基线，重点解释配置的运行时语义与工程组合方式。

由于 OpenClaw 迭代较快，某些字段默认值可能在不同文档语言版本或不同发布日期之间发生变化。本手册对发现的版本敏感项进行显式标注，例如 Code Mode 当前英文文档显示默认激活层为 ``"auto"``，而较早/缓存中文页面仍可能出现“默认关闭”的描述。生产环境应以本机实时 Schema 为准。

阅读地图

阅读目标	重点章节	你会得到什么
快速看懂 openclaw.json	第 1-4 章	总架构、根字段、继承覆盖、配置加载
搭 Coding/Research Agent	第 5-10 章	Context、Model、Tool、Sandbox、MCP/Skill/Plugin
做 Multi-Agent 与渠道路由	第 11-12 章	Session、Messages、Bindings、Channels
生产部署与安全	第 13-16 章	Gateway、Secrets、Automation、Observability、Hardening
直接落地	第 17-19 章 + 附录	模板、诊断流程、字段检查清单

目录（章节索引）

- 1. 1. openclaw.json 的真正角色：声明式 Agent Runtime 控制面
- 2. 2. 配置加载、JSON5、Schema 与热重载

3. 3. 双分区模型与覆盖优先级
 4. 4. 根字段总览：从基础设施到 Agent Loop
 5. 5. agents：Agent Runtime 的核心对象
 6. 6. Context Engineering：Bootstrap、Startup Context、Limits、Compaction、Pruning
 7. 7. Model Routing：primary、fallback、utility、image、pdf、media
 8. 8. Tools：Profile、Group、Allow/Deny、Provider/Sender Policy
 9. 9. Code Mode：把 Tool Schema 从上下文里移走
 10. 10. Sandbox 与 Elevated：授权和隔离为何必须分开
 11. 11. MCP、Skills 与 Plugins：能力注册、渐进披露、插件槽位
 12. 12. Session、Messages、Bindings、Channels：人、会话与 Agent 路由
 13. 13. Gateway：服务边界、Auth、TLS、Reload 与 Control UI
 14. 14. Env、Auth、Secrets：凭据生命周期与 SecretRef
 15. 15. Browser、Hooks、Cron、Heartbeat：长期运行 Agent 的外部世界
 16. 16. Logging 与 Diagnostics：Production Observability
 17. 17. 三套推荐配置：Coding / Research / Multi-Agent
 18. 18. 安全加固 Checklist 与常见误配
 19. 19. CLI 诊断与升级维护 workflow
- 附录 A：关键字段速查表
 - 附录 B：配置命令速查
 - 附录 C：官方参考资料

1. openclaw.json 的真正角色：声明式 Agent Runtime 控制面

“~/openclaw/openclaw.json”不只是模型 API 配置，而是 Gateway、Agent Runtime、Context、Tool、Sandbox、Session、Channel、MCP、Skill、Plugin、Browser、Secrets、Automation 与 Observability 的统一控制面。

```
openclaw.json
├── agents      Agent Loop / Context / Model / Sandbox
├── models      Provider 与模型目录
├── tools       Tool capability + policy
├── mcp         外部 MCP Server
├── skills      Skill 能力包
├── plugins     Runtime Plugin
├── memory      Memory 搜索/索引/引用
├── session     会话生命周期
├── messages    并发消息与输出策略
├── channels    Telegram / Discord / Slack / WhatsApp ...
├── bindings    Channel / Peer → Agent Router
├── browser     Browser runtime / CDP
├── gateway     OpenClaw server
├── env/auth/secrets Credential plane
├── hooks/cron   Webhook + Automation
└── logging/diagnostics Observability
```

1.1 为什么叫“控制面”

控制面不直接完成任务，而是声明“系统允许哪些能力、由谁运行、上下文如何构建、动作在哪里执行、会话如何持续、外部事件如何进入”。每个 Agent turn 都是在这组策略约束下被装配出来。

```
Infrastructure / Routing
  ↓
Agent Defaults
  ↓
Per-Agent Override
  ↓
Session + Sender + Channel
  ↓
Context Builder
  ↓
Model Router
  ↓
Effective Tool Policy
  ↓
```

Sandbox / Host Execution



Tool Result → Agent Loop

Know-why OpenClaw 的价值不在于提供更多字段，而在于把“能力、身份、上下文、授权、隔离、生命周期”从 Agent 代码里抽离为可审计、可热更新、可继承的声明式策略。

2. 配置加载、JSON5、Schema 与热重载

2.1 配置文件与 JSON5

默认路径为 `~/.openclaw/openclaw.json`。配置格式是 JSON5，因此允许注释和尾随逗号。文件缺失时，OpenClaw 使用安全默认值。官方同时提醒：有效配置路径应是普通文件；OpenClaw 自己写配置时会原子替换目标，因此不建议用 symlink 布局。

```
// ~/.openclaw/openclaw.json
{
  agents: {
    defaults: {
      workspace: "~/.openclaw/workspace",
    },
  },
}
```

2.2 Schema：网页不是最终真相

- `openclaw config schema`：打印当前安装版本用于校验和 Control UI 的实时 JSON Schema。
- `config.schema.lookup`：获取某个精确路径的 Schema 节点，适合 Agent 在修改配置前做字段级验证。
- 插件与 Channel 可以把自己的配置 Schema 合并进实时 Schema，因此“完整字段全集”随安装环境变化。
- 未知字段、错误类型或非法枚举可能导致配置校验失败，Gateway 甚至拒绝启动。

```
openclaw config schema > openclaw.schema.json
openclaw config validate
openclaw config validate --json
```

2.3 `\$include`：大配置的模块化

```
{
  gateway: { port: 18789 },
  agents: { $include: "./agents.json5" },
  plugins: { $include: "./plugins.json5" },
}
```

单文件 include 会替换包含位置的对象；数组 include 会按顺序深度合并，后者覆盖前者。官方文档给出的最大嵌套深度为 10 层。

推荐 超过约 300-500 行的配置，建议拆成 `agents.json5`、`models.json5`、`channels.json5`、`plugins.json5` 等，并保留一个最小 root 文件。

3. 双分区模型与覆盖优先级

OpenClaw 官方把配置描述为“两桶规则”：根级同级项主要承载基础设施与跨 Agent 行为，`agents.defaults` 主要承载 Agent Loop 行为；Schema 支持的字段可以在 `agents.entries.<id>` 中做 per-agent override。

```
Root infrastructure
(models / tools / gateway / channels / session / plugins ...)
    ↓
agents.defaults
    ↓
agents.entries.<id>
    ↓
channel / sender / session effective policy
    ↓
runtime
```

3.1 继承与覆盖的工程意义

把公共配置放进 defaults，把差异留给 entries，可以避免几十个 Agent 复制一整套配置。配置 review 时也更容易识别“这是全局安全边界，还是某个 Agent 的特例”。

```
agents: {
  defaults: {
```

```
    model: "anthropic/claude-sonnet-4-6",
    thinkingDefault: "low",
  },
  entries: {
    coding: {
      model: "openai/gpt-5.4",
      thinkingDefault: "high",
    },
  },
}
```

常见坑 并非所有数组字段都会 merge。有些 allowlist/skills 列表在 per-agent 层是 replacement 语义。升级或新增字段前，先查该路径的实时 Schema 与官方说明。

4. 根字段总览：从基础设施到 Agent Loop

根字段	主要职责	控制面类别	重要性
env	环境变量注入、shellEnv	基础设施	中
auth	Provider credential profile 路由	安全	高
secrets	SecretRef / provider	安全	高
models	Provider、模型目录、Base URL	模型	最高
agents	Agent defaults / entries	Agent runtime	最高
bindings	Channel/Peer → Agent	路由	高
channels	聊天渠道、DM/Group access	入口	高
session	会话隔离、reset、maintenance	生命周期	最高
messages	队列、ack、可见回复	并发/交互	中
tools	工具 profile 与 policy	授权	最高

根字段	主要职责	控制面类别	重要性
mcp	出站 MCP server registry	能力	高
skills	Skill discovery/限制	能力/上下文	高
plugins	插件加载与 slot	扩展	高
browser	Browser/CDP/SSRF	执行面	高
gateway	服务监听/Auth/TLS/Reload	基础设施	最高
hooks	Webhook → Agent	自动化	中
cron	调度与 failure alert	自动化	高
logging	日志与 audit	可观测	中
diagnostics	OTel traces/metrics/logs	可观测	中
update	更新 channel/auto	运维	低
acp	外部 Agent runtime bridge	扩展	中
ui/desktop/talk	人机交互相关	交互	低-中

版本敏感 ``channels.*``、``plugins.entries.*.config`` 等深层字段可以由插件提供，不能只靠一张静态字段表判断当前环境是否支持。

5. agents：Agent Runtime 的核心对象

5.1 ``agents.defaults.workspace``

Workspace 是 Agent 的工作目录，也是长期上下文与项目资产的聚合点。官方默认通常是 ``~/openclaw/workspace``；使用非默认 profile 时可能带 profile 后缀。多 Agent fleet 中，没有独立 workspace 的 Agent 会被隔离到 agent-id 子目录。


```
agents: {
  defaults: {
    workspace: "~/.openclaw/workspace",
  },
}
```

Workspace 里常见的引导文件包括 `AGENTS.md`、`BOOTSTRAP.md`，可选文件可包括 `SOUL.md`、`USER.md`、`IDENTITY.md`。不同版本对默认生成文件存在演进，因此仍应以当前 Schema/文档为准。

5.2 `agents.entries`：一个 Agent 不是一个 Prompt

```
agents: {
  entries: {
    main: {
      name: "Main",
      workspace: "~/.openclaw/workspace-main",
      model: "anthropic/claude-sonnet-4-6",
      utilityModel: "openai/gpt-5.6-luna",
      thinkingDefault: "high",
      identity: { name: "Clawd", theme: "helpful", emoji: "🦜" },
      tools: { profile: "coding" },
    },
  },
}
```

工程上，一个 OpenClaw Agent 应被理解为：Model + Context Policy + Workspace + Identity + Skills + Tools + Sandbox + Session + Runtime，而不是 `system prompt + model`。

5.3 Subagents

Subagent 配置控制子任务并发、可 spawn 的 Agent、最大深度、超时与归档。其目的不是制造无限递归，而是把可并行、可隔离的子问题从主 Context 中移走。

典型字段	作用	设计原因
allowAgents	限制可 spawn 的 Agent	降低任意委派风险
maxConcurrent	控制并发 child run	限流与成本控制
maxChildrenPerAgent	单 session 子任务上限	防止 fan-out 爆炸
maxSpawnDepth	最大递归层级	避免 agent tree 无界增长

典型字段	作用	设计原因
runTimeoutSeconds	子任务超时	故障隔离
archiveAfterMinutes	历史 child run 归档	生命周期管理

6. Context Engineering：Bootstrap、Startup Context、Limits、Compaction、Pruning

OpenClaw 把 Context Window 视为有限的工作内存，而不是无限知识库。其配置因此围绕“何时注入、注入多少、何时检索、何时压缩、哪些结果应该裁剪”展开。

6.1 `contextInjection`

值	语义	适用场景
always	每轮重新注入 workspace bootstrap	最稳妥；配置变化需立即生效
continuation-skip	安全续接轮次跳过重复注入	长对话降 token；Heartbeat/压缩恢复仍重建
never	禁用 workspace/bootstrap 注入	自定义 Context Engine 或完全自行构建 prompt

```
agents: {
  defaults: {
    contextInjection: "continuation-skip",
    bootstrapMaxChars: 20000,
    bootstrapTotalMaxChars: 60000,
  },
}
```

6.2 Bootstrap Budget

`bootstrapMaxChars` 限制单个 workspace bootstrap 文件的最大注入字符；`bootstrapTotalMaxChars` 限制所有 bootstrap 合计。官方当前文档给出的 defaults 分别是 20,000 与 60,000 字符。

Know-how 把稳定规则放在 Bootstrap，把短期状态放到 Startup Context / Memory。不要让 AGENTS.md 变成“所有知识都塞进去”的垃圾桶。

6.3 `startupContext`

```
startupContext: {
  enabled: true,
  applyOn: ["new", "reset"],
  dailyMemoryDays: 2,
  maxFileBytes: 16384,
  maxFileChars: 1200,
  maxTotalChars: 2800,
}
```

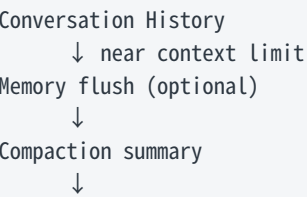
Startup Context 解决的是“新 Session 第一轮需要恢复最近状态”的问题：例如最近几天 Memory、未完成任务、今天的工作上下文。它与长期稳定的 Bootstrap 分层。

6.4 `contextLimits`

`memoryGetMaxChars` 限制默认 `memory\_get` 摘录；`postCompactionMaxChars` 限制压缩恢复时重新注入的 AGENTS.md 摘录。官方当前还会根据模型 Context Window 动态限制 live tool result：较小窗口约 16K chars，100K+ 时约 32K，200K+ 时约 64K。

6.5 Compaction 与 Pruning 的区别

机制	处理对象	是否总结	是否改变磁盘历史	目的
Compaction	对话历史	是	通常保留原始持久历史/以运行时策略为准	把旧历史压成高密度摘要
Context Pruning	旧 tool result / context surface	通常否	否	减少无价值大结果再次进入模型



Preserve recent turns
↓
Continue Agent Loop

Know-why 高质量长运行 Agent 的关键不是“拥有更大的 Context”，而是“让正确的信息在正确阶段进入 Context，并让低价值中间数据及时离开”。

7. Model Routing：primary、fallback、utility、image、pdf、media

7.1 `agents.defaults.model` vs 根级 `models`

路径	回答的问题	例子
agents.defaults.model	这个 Agent 本轮主要用哪个模型？	primary + fallbacks
models / providers	系统里有哪些 Provider/模型，如何连接？	baseUrl、api、apiKey、模型 catalog

```
agents: {
  defaults: {
    model: {
      primary: "anthropic/claude-sonnet-4-6",
      fallbacks: ["openai/gpt-5.4"],
    },
  },
}
```

7.2 `utilityModel`

官方当前把 `utilityModel` 定义为低成本内部任务模型，用于仪表板 session title、部分 Channel thread/topic title、progress narration 等。per-agent `agents.entries.\*.utilityModel` 可覆盖 defaults。未显式设置时，OpenClaw 可以根据主 Provider 的 small-model default 自动选择；若没有则回退主模型。

Know-why 不要让最昂贵的推理模型做“给会话起标题”这样的 housekeeping。Production Agent 应把能力分层：复杂 reasoning 用主模型，短内部任务用 utility model。

7.3 多模态专用路由

- ``imageModel``：当主模型不能接受图像时的图像理解 fallback。
- ``pdfModel``：``pdf`` tool 使用；未设时可回退到 `imageModel`，再到 `session/default model`。
- ``mediaModels.{image,music,video}``：共享媒体生成工具的 `provider/model` 路由。

7.4 自定义 Provider

```
models: {
  mode: "merge",
  providers: {
    local: {
      baseUrl: "http://127.0.0.1:8080/v1",
      apiKey: "${LOCAL_API_KEY}",
      api: "openai-completions",
      models: [
        {
          id: "qwen-local",
          name: "Qwen Local",
          input: ["text", "image"],
          contextWindow: 131072,
          maxTokens: 32000,
        },
      ],
    },
  },
}
```

``models.mode: "merge"`` 适合在内置 catalog 基础上增加自定义 Provider；``replace`` 更适合完全自己维护 model catalog 的场景。

8. Tools : Profile 、 Group 、 Allow/Deny 、 Provider/Sender Policy

8.1 Tool Profile

Profile	核心含义	建议
minimal	仅最基本 session 状态能力	极低风险信息型 Agent
coding	FS + runtime + web + sessions + memory + automation 等	Coding/Research 默认起点
messaging	消息与 session 编排	客服/通知/协作 Agent
full	不施加 profile 限制	只建议受控开发环境

当前官方 Tools 文档说明：本地 onboarding 在未设置 profile 时，会为新本地配置默认写入 `tools.profile: "coding"`。

8.2 Tool Group

Group	典型工具
group:runtime	exec / process / code_execution
group:fs	read / write / edit / apply_patch
group:sessions	sessions_* / conversations_* / subagents
group:memory	memory_search / memory_get
group:web	web_search / x_search / web_fetch
group:ui	browser / screen / terminal / canvas / show_widget
group:automation	heartbeat_respond / cron / gateway
group:messaging	message

Group	典型工具
group:nodes	nodes / computer
group:agents	agents_list / goal / plan / ask_user / skill_workshop
group:plugins	加载的插件工具与 bundle-mcp

8.3 `allow` / `deny` : deny wins

```
tools: {
  profile: "coding",
  allow: ["group:fs", "group:web"],
  deny: ["exec"],
}
```

Tool Policy 是授权层：某个工具是否对 Agent 可见、是否允许调用。它与 Sandbox 的“在哪里执行”是不同维度。

8.4 Provider / Sender 级 Policy

`tools.byProvider` 允许针对不同模型/Provider 限制工具；sender 级 policy 则可以进一步把 Owner、Guest、不同 Channel 身份映射到不同权限。

Know-how 把“模型能力差异”和“用户信任等级差异”都落到 Tool Policy，而不是塞进 prompt 里说“请不要调用危险工具”。Prompt 不是访问控制系统。

9. Code Mode : 把 Tool Schema 从上下文里移走

Code Mode 是 OpenClaw 当前非常重要的 Tool orchestration 机制：当它参与某次 run 时，模型不再看到全部启用工具 schema，而主要看到 `exec`、`wait` 和少量必须 direct-call 的工具；模型生成 JavaScript/TypeScript，通过隐藏 catalog 搜索、描述并调用工具。

```
普通 Tool Calling
LLM → Tool schema A/B/C/... → call → result → LLM → call ...
```

```

Code Mode
LLM → exec(code)
    ├── search tools
    ├── describe tool
    ├── call tool A
    ├── call tool B in parallel
    └── filter / join / transform
    → compact result → LLM

```

9.1 当前默认值的版本变化

版本敏感 当前英文 Code Mode 文档显示 `tools.codeMode.enabled` 默认是 `"auto"`：仅对 catalog 标记为 preferred code-mode performer 的模型自动启用；旧/缓存文档可能仍显示“默认关闭”。请以实时 Schema 为准。

```

tools: {
  codeMode: {
    enabled: "auto", // false | true | "auto"
    timeoutMs: 10000,
    memoryLimitBytes: 67108864,
    maxOutputBytes: 65536,
    maxPendingToolCalls: 16,
  },
}

```

9.2 为什么它重要

- 减少几十/几百个 Tool Schema 常驻 System/Tool Context 的成本。
- 把循环、并行、条件、过滤、聚合放进一次代码单元，而不是让模型来回多轮调工具。
- 对 OpenClaw tool、Plugin tool、MCP tool 使用统一 orchestration 形态。
- 失败时采用 fail-closed 语义：已激活 Code Mode 但 QuickJS-WASI 不可用时，不会静默暴露全部普通工具。

安全上应把模型生成的代码视为不可信代码。Code Mode 自己不改变 Tool Authorization；有效工具集先经过 agent/model/provider/sandbox/channel/sender policy，之后才进入 Code Mode catalog。

10. Sandbox 与 Elevated：授权和隔离为何必须分开

Tool Policy = 是否有资格调用能力
Sandbox = 被允许的能力在哪里执行
Elevated = 是否允许越过 sandbox 到 host / node

10.1 `sandbox.mode` 与 workspace access

字段/值	意义	建议
mode: off	不使用 sandbox	仅受信任的本机开发
mode: non-main	非主 session 隔离	兼顾主会话便利与子任务隔离
mode: all	所有 eligible run 隔离	多用户/外部输入更稳妥
workspaceAccess: none	看不到真实 workspace	不可信任任务
workspaceAccess: ro	只读	Research/Review
workspaceAccess: rw	读写	Coding，但配合 repo/FS 边界

10.2 为什么 deny + sandbox 要同时存在

只配置 Sandbox 等于“允许调用，但限制执行环境”；只配置 Tool deny 等于“根本不让模型拿到这个能力”。Production 安全通常需要两层同时存在：Authorization + Isolation。

安全原则 对只读 Agent，仅 `workspaceAccess: ro` 不够；还应 deny mutating FS tools 与必要的 runtime 工具，避免通过脚本或间接路径产生写入。

10.3 Elevated

Elevated 是高权限 escape hatch：让符合来源策略的执行离开 sandbox。应默认关闭，只对明确的 owner/sender 开放，并结合审计与强认证。

11. MCP、Skills 与 Plugins：能力注册、渐进披露、插件槽位

11.1 MCP：外部 Tool Server 注册表

```
mcp: {
  servers: {
    docs: {
      command: "npx",
      args: ["-y", "@modelcontextprotocol/server-fetch"],
    },
    remote: {
      url: "https://example.com/mcp",
      transport: "streamable-http",
      requestTimeoutMs: 20000,
      connectionTimeoutMs: 5000,
      supportsParallelToolCalls: true,
      headers: { Authorization: "Bearer ${MCP_REMOTE_TOKEN}" },
      toolFilter: {
        include: ["search_*"],
        exclude: ["admin_*"],
      },
    },
  },
}
```

MCP 有两个完全不同的方向：`openclaw mcp serve` 是 OpenClaw 作为 MCP Server；`mcp.servers` 是 OpenClaw 作为 MCP Client-side registry，把第三方 MCP 工具投影给其 runtime。

11.2 MCP + Sandbox 的“双门”

OpenClaw-managed MCP 工具由 `bundle-mcp` 插件拥有。正常 tool profile 可以允许它们，但 sandboxed session 还有一层 `tools.sandbox.tools` gate。服务器成功加载不等于工具一定出现在模型请求里。

```
tools: {
  sandbox: {
    tools: {
      alsoAllow: ["bundle-mcp"],
    },
  },
}
```

典型故障 MCP status 正常、server 能连接，但 sandbox run 中模型看不到 MCP 工具：优先检查 ``tools.sandbox.tools.alsoAllow`` 是否允许 ``bundle-mcp``、``group:plugins`` 或对应 server glob。

11.3 Skills：按需披露领域知识

Skill 更接近“可渐进加载的领域操作包”，而不是常驻 Tool Schema。工程目标是减少 System Prompt 常驻知识，通过 catalog + 按需读取 SKILL.md / 资源实现 Progressive Disclosure。

11.4 Plugins：进程级扩展与独占槽位

Plugin 可以提供 Channel、Tool、Provider、MCP Server、Context Engine、Memory 等能力。

``plugins.deny`` 优先于 allow 和 per-plugin enablement；workspace-origin plugin 默认更谨慎。

```
plugins: {
  allow: ["my-plugin"],
  entries: {
    "my-plugin": { enabled: true, config: {} },
  },
  slots: {
    memory: "memory-core",
    contextEngine: "legacy",
  },
}
```

``plugins.slots.memory`` 与 ``plugins.slots.contextEngine`` 属于独占槽位：同一类别只选一个实现。这说明 Memory/Context Engine 在 OpenClaw 架构里本身就是可插拔 runtime component。

12. Session、Messages、Bindings、Channels：人、会话与 Agent 路由

12.1 Session 隔离

Session 配置决定不同 sender、DM、group 是否共享上下文，以及何时 reset、如何维护历史。多用户部署最关注的是避免把不同人的私聊意外汇入同一个上下文。

```
session: {
  dmScope: "per-peer",
```

```
groupScope: "per-group",
reset: {
  mode: "idle",
  idleMinutes: 240,
},
maintenance: {
  mode: "enforce",
  pruneAfter: "30d",
  maxEntries: 500,
},
}
```

12.2 Messages：长运行任务的人机并发协议

Queue 模式	语义	适用
steer	新消息影响当前 run	交互式长任务
followup	新消息排到下一 turn	需要保证当前步骤完成
collect	先聚合再处理	连续输入/多段内容
interrupt	新消息中断当前工作	高优先级人类控制

因此 `messages.queue` 不只是 UI 设置，而是“用户在 Agent 运行期间继续说话时，runtime 如何处理并发”的协议。

12.3 Bindings：声明式 Multi-Agent Router

```
bindings: [
  {
    agentId: "home",
    match: { channel: "whatsapp", accountId: "personal" },
  },
  {
    agentId: "work",
    match: { channel: "whatsapp", accountId: "biz" },
  },
]
```

Binding 把 Channel/account/peer/guild/team 等外部身份映射到 Agent。Multi-Agent 系统中，应把“哪个入口去哪个 Agent”作为配置路由，而不是在 prompt 中做模糊判断。

12.4 Channel DM / Group access

策略	行为	建议
DM pairing	陌生 sender 先配对	默认安全入口
DM allowlist	仅 allowFrom	固定用户/机器人入口
DM open	所有人	仅公开服务且强 Tool/Sandbox 限制
DM disabled	关闭私聊	只做群组/事件入口
Group allowlist	只允许指定 group	默认推荐
Group open	所有 group	需要 mention gate 与 sender policy

官方当前文档说明所有 Channel 都支持 DM / group policy 的共同模型，但各 Channel 的深层字段由核心或插件 Schema 扩展。

13. Gateway：服务边界、Auth、TLS、Reload 与 Control UI

13.1 Gateway 是什么

Gateway 是 OpenClaw 的 WebSocket 服务与运行时中心，连接 Channel、Node、Session、Hook 等。
`gateway.mode=local` 才允许以正常本地 Gateway 方式启动；官方 CLI 文档说明未配置时会拒绝正常启动，除非显式使用开发型 bypass。

```
gateway: {
  mode: "local",
  port: 18789,
  bind: "loopback",
  auth: {
    mode: "token",
    token: "${OPENCLAW_GATEWAY_TOKEN}",
  },
  reload: { mode: "hybrid" },
}
```

13.2 `bind` 与攻击面

bind	含义	风险
loopback	只监听本机	最低
lan	对局域网暴露	必须强认证 + TLS/网络边界
tailnet	面向 tailnet	仍应保留应用层 auth
custom	自定义地址	按实际网络边界评估

安全底线 从 loopback 改到任何非本机监听，本质上是在扩大攻击面；不要把“内网”当成身份验证。

13.3 Auth / TLS / Reload

- Auth 模式可包括 token、password、trusted-proxy 等；默认生产建议使用显式认证。
- TLS 可以使用证书路径，也可在本地/开发场景自动生成自签证书。
- `reload.mode: "hybrid"` 会对可热更新字段直接应用，对需要重启的变更自动重启。
- Control UI 从实时 Schema 渲染配置表单，因此插件/Channel 字段可动态出现在 UI 中。

14. Env、Auth、Secrets：凭据生命周期与 SecretRef

14.1 不要把 Secret 明文长期写在 config

```
apiKey: "${OPENAI_API_KEY}"
```

`\${VAR}` 适合常规环境变量替换。更结构化的方式是 SecretRef，由 provider 负责解析 secret。

```
apiKey: {
  source: "env",
  provider: "default",
  id: "OPENAI_API_KEY",
}
```

14.2 Secret Provider

```
secrets: {
  providers: {
    default: { source: "env" },
    filemain: {
      source: "file",
      path: "~/.openclaw/secrets.json",
      mode: "json",
    },
  },
},
}
```

SecretRef 的核心是把“配置引用”与“真实 Secret 值”分离。生产环境还可以使用 exec/store 类 provider，把 Vault/Password manager/企业 secret service 接入到解析层。

14.3 Auth Profile

`auth.profiles` 更像 credential profile 的元数据与路由；实际 Agent-scoped credential 通常在 Agent 目录下的 auth profile 存储中维护。这样一个配置可以声明 Provider 与 profile 选择，而不把全部敏感材料放在 root JSON。

15. Browser 、 Hooks 、 Cron 、 Heartbeat ：长期运行 Agent 的外部世界

15.1 Browser

Browser 可以连接 OpenClaw 管理的浏览器、已有 session、Extension 或 remote CDP。安全上重点关注：是否允许 evaluate、是否 attach only、是否允许 private network、是否有 SSRF allowlist。

```
browser: {
  enabled: true,
  evaluateEnabled: false,
  defaultProfile: "openclaw",
  profiles: {
    openclaw: {},
  },
},
}
```

Know-why Browser 是一个高权限外部执行面：它能携带登录态、访问内部站点、执行页面 JS。把 SSRF、

private-network 与 evaluate 当成安全边界，而不是“浏览器小功能”。

15.2 Hooks

Hooks 把 Webhook 映射到 Agent action。外部 Webhook 是典型“不可信输入”，推荐使用专门的 restricted Agent，并限制 tools、sandbox、allowedAgentIds 和可持久化 session 行为。

15.3 Cron / Heartbeat

Cron 提供计划任务和失败告警；Heartbeat 则更像周期性唤醒 Agent 进行状态检查。长期运行 Agent 不是“while true”，而是由可审计调度器管理唤醒、隔离 session、超时和通知。

```
heartbeat: {
  every: "30m",
  model: "openai/gpt-5.6-luna",
  timeoutSeconds: 45,
  lightContext: true,
}
```

`lightContext` 的意义是避免每次轻量健康检查都加载完整 workspace bootstrap，降低 token 与延迟。

16. Logging 与 Diagnostics : Production Observability

16.1 Logging / Audit

```
logging: {
  level: "info",
  consoleLevel: "info",
  audit: {
    enabled: true,
    messages: "off",
  },
}
```

Audit 更适合作为 Execution Ledger：谁、何时、调用了什么、结果状态如何；敏感 prompt/message/tool args 是否记录，应单独审慎开启。

16.2 OpenTelemetry

```
diagnostics: {
  enabled: true,
  otel: {
    enabled: true,
    endpoint: "https://otel.example.com",
    traces: true,
    metrics: true,
    logs: false,
    sampleRate: 0.1,
    captureContent: false,
  },
},
}
```

对于 Agent 系统，Trace 应至少能回答：这次 run 使用了哪个 Agent/模型、Context 多大、调用了哪些工具、每步耗时、是否 sandbox、失败在哪里、fallback 是否发生。`captureContent: false` 是降低敏感内容泄露的重要默认。

17. 三套推荐配置：Coding / Research / Multi-Agent

17.1 Coding Agent：主会话可写、子任务隔离

```
{
  gateway: {
    mode: "local",
    bind: "loopback",
    auth: { mode: "token", token: "${OPENCLAW_GATEWAY_TOKEN}" },
  },
  agents: {
    defaults: {
      workspace: "~/openclaw/workspace",
      model: {
        primary: "anthropic/claude-sonnet-4-6",
        fallbacks: ["openai/gpt-5.4"],
      },
      contextInjection: "continuation-skip",
      bootstrapMaxChars: 20000,
      bootstrapTotalMaxChars: 60000,
      sandbox: {
        mode: "non-main",
        workspaceAccess: "rw",
      },
    },
  },
  entries: {
    main: {
      tools: { profile: "coding" },
    },
  },
}
```

```

    },
  },
},
tools: {
  profile: "coding",
  codeMode: { enabled: "auto" },
  elevated: { enabled: false },
},
session: {
  dmScope: "per-peer",
  groupScope: "per-group",
},
}

```

设计思路：主 Agent 有完整 coding profile；非主任务进 sandbox；默认不 elevated；Context 用 continuation-skip 减少重复；Code Mode 交给模型 catalog 自动判断。

17.2 Research Agent：只读 + Web/Memory + 强隔离

```

agents: {
  entries: {
    research: {
      thinkingDefault: "high",
      tools: {
        profile: "coding",
        deny: ["write", "edit", "apply_patch", "group:runtime"],
      },
      sandbox: {
        mode: "all",
        workspaceAccess: "ro",
      },
    },
  },
},
}

```

设计思路：Research 的“只读”应同时从 Tool Policy 与 Sandbox FS 两层实现；不要只依赖 prompt 中的一句“不要修改文件”。

17.3 Multi-Agent：入口路由 + 独立 workspace

```

agents: {
  entries: {
    home: { workspace: "~/openclaw/workspace-home" },
    work: { workspace: "~/openclaw/workspace-work" },
  },
},
bindings: [

```

```
{ agentId: "home", match: { channel: "whatsapp", accountId: "personal" } },
{ agentId: "work", match: { channel: "whatsapp", accountId: "biz" } },
],
session: {
  dmScope: "per-peer",
  groupScope: "per-group",
}
```

设计思路：外部身份先经过 Binding 决定 Agent，再由 Session scope 隔离不同用户/群组；不要让所有入口都落在共享主 Agent 再靠 LLM 猜“这是工作还是生活”。

18. 安全加固 Checklist 与常见误配

18.1 Production 最小安全基线

层	推荐动作	为什么
Gateway	bind 尽量 loopback；非 loopback 必须强 auth/TLS/网络边界	避免服务面裸露
Secrets	使用 env/SecretRef；避免明文 token	减少配置泄漏风险
Channels	DM pairing/allowlist；Group allowlist/mention gate	限制不可信入口
Session	多用户用 per-peer/per-group	防止跨用户上下文泄漏
Tools	profile + allow/deny；deny mutating/runtime as needed	模型不可把 prompt 当 ACL
Sandbox	外部输入/子 Agent 使用 all 或 non-main	限制执行 blast radius
Elevated	默认 false；只给 owner	避免 sandbox 绕过
MCP	toolFilter + sandbox second gate	缩小 tool surface
Browser	evaluate/private network/SSRF 受限	防止浏览器成为内网代理
Hooks	专用 restricted Agent	Webhook 是不可信输入
Logging	audit on；content capture 谨慎	可审计且不泄密

层	推荐动作	为什么
Update	生产升级前 validate + schema diff	防止默认值/字段演进破坏行为

18.2 十个常见误配

- 20. 认为 `sandbox.mode=all` 就等于“安全”，却没有限制 Tool allow/deny。
- 21. 把 owner 与普通 sender 放进同一个 DM session scope。
- 22. 把 MCP Server 配上了，却忘记 sandbox 的 `bundle-mcp` 第二道 allow gate。
- 23. 把所有 Provider/API Key 明文写进 openclaw.json 并提交到 Git。
- 24. 把 Gateway 从 loopback 改成 LAN，却仍使用弱认证或无 TLS/网络边界。
- 25. Research Agent 只设置 workspace ro，却保留 exec/process 等可变更系统状态的 runtime 工具。
- 26. AGENTS.md 无限膨胀，把历史、记忆、流程、资料全部塞进 bootstrap。
- 27. 每个 Agent 复制整套配置，造成安全策略逐渐漂移。
- 28. 根据博客/旧截图照抄字段，不先跑 `openclaw config schema`。
- 29. 升级后发现行为变化，只排查 prompt，不检查默认值、schema、tool profile 与 plugin slot 是否变化。

19. CLI 诊断与升级维护工作流

19.1 每次配置修改前后

```
# 1. 确认实际文件
openclaw config file

# 2. 导出实时 Schema
openclaw config schema > openclaw.schema.json

# 3. 读取关键值
openclaw config get agents.defaults.model --json

# 4. 修改
openclaw config set agents.defaults.heartbeat.every "2h"

# 5. 校验
openclaw config validate
openclaw config validate --json

# 6. 升级/修复旧字段
openclaw doctor --fix
```

19.2 MCP 诊断

```
openclaw mcp list
openclaw mcp status --verbose
openclaw mcp doctor
openclaw mcp probe <server>
```

19.3 建议的变更流程

- 30. 备份当前 openclaw.json 与相关 include 文件。
- 31. 导出当前 `openclaw config schema`，记录 OpenClaw 版本。
- 32. 只改一个控制面主题，例如先改 Tool Policy，不同时改 Session + Model + MCP。
- 33. 执行 `config validate`，必要时 `doctor --fix`。
- 34. 对关键 Agent 做最小回归：模型选择、Tool 可见列表、Sandbox、MCP、Session 隔离。
- 35. 观察日志/trace，确认 fallback、code mode、tool filtering 是否符合预期。
- 36. 再提交配置变更。

Know-how 把 openclaw.json 当基础设施代码（IaC）管理：版本控制、review、schema validation、变更审计、回滚，而不是在生产机上手改到“能跑就行”。

附录 A：关键字段速查表

路径	作用	控制面	优先级
agents.defaults.workspace	Agent 工作区	Context + Filesystem	常用
agents.defaults.contextInjection	Bootstrap 注入时机	Context	关键
agents.defaults.bootstrapMaxChars	单文件注入预算	Context	关键
agents.defaults.bootstrapTotalMaxChars	总注入预算	Context	关键

路径	作用	控制面	优先级
agents.defaults.startupContext	新/重置 Session 最近状态	Context	中高
agents.defaults.contextLimits	运行时摘录上限	Context	中高
agents.defaults.model	主模型与 fallback	Model	关键
agents.defaults.utilityModel	内部短任务模型	Cost/Latency	中
agents.entries.*	Per-agent override	Agent	关键
tools.profile	基础工具 allowlist	Authorization	关键
tools.allow / deny	工具增删权限	Authorization	关键
tools.byProvider	按 Provider/Model 限工具	Authorization	中高
tools.codeMode	代码编排模式	Context/Tool	中高
agents.*.sandbox	执行隔离	Isolation	关键
tools.elevated	越过 sandbox 的高权限	Isolation/Auth	高风险
mcp.servers	MCP registry	Capability	高
mcp.*.toolFilter	MCP tool filtering	Authorization/Context	高
skills.*	Skill discovery/限制	Capability/Context	中高
plugins.allow/deny	插件 allowlist	Capability	高
plugins.slots.*	独占 runtime implementation	Runtime	中高
session.dmScope/groupScope	会话隔离	Privacy	关键
session.reset/maintenance	会话生命周期	Lifecycle	中高
messages.queue	运行中消息并发策略	Interaction	中
bindings	外部入口 → Agent	Routing	关键
channels.*	Channel access/policy	Ingress	关键

路径	作用	控制面	优先级
gateway.bind	监听范围	Network	关键
gateway.auth	Gateway 身份验证	Security	关键
gateway.reload	配置热更新	Ops	中
secrets.providers	Secret backend	Security	高
browser.*	Browser runtime/SSRF	Execution	高
hooks.*	Webhook routing	Automation	高
cron.*	调度	Automation	中高
logging.audit	执行审计	Observability	高
diagnostics.otel	Trace/metrics/logs	Observability	中高

附录 B：配置命令速查

命令	用途
openclaw onboard	完整首次引导
openclaw configure	配置向导
openclaw config file	显示实际配置文件
openclaw config schema	输出实时 Schema
openclaw config get <path>	读取字段
openclaw config set <path> <value>	修改字段
openclaw config unset <path>	删除字段
openclaw config validate	校验当前配置

命令	用途
openclaw doctor --fix	修复/迁移可识别旧配置
openclaw gateway	启动 Gateway
openclaw channels list	查看 Channel
openclaw mcp status --verbose	检查 MCP registry/status
openclaw mcp doctor	MCP 配置诊断
openclaw mcp probe <server>	实际连接探测 MCP

附录 c：官方参考资料

以下资料用于本手册的字段基线与运行时解释。OpenClaw 迭代较快，阅读时应优先查看页面最近更新时间，并最终与本机 `openclaw config schema` 对照。

编号	文档	地址
S1	Configuration	https://docs.openclaw.ai/gateway/configuration
S2	Configuration Reference	https://docs.openclaw.ai/gateway/configuration-reference
S3	Configuration - Agents	https://docs.openclaw.ai/gateway/config-agents
S4	Configuration - Tools and Custom Providers	https://docs.openclaw.ai/gateway/config-tools
S5	Configuration - Channels	https://docs.openclaw.ai/gateway/config-channels
S6	Code Mode	https://docs.openclaw.ai/tools/code-mode
S7	Models	https://docs.openclaw.ai/concepts/models

编号	文档	地址
S8	MCP CLI / concepts	https://docs.openclaw.ai/cli/mcp
S9	Plugins	https://docs.openclaw.ai/tools/plugin
S10	Gateway CLI	https://docs.openclaw.ai/cli/gateway
S11	Configuration Examples	https://docs.openclaw.ai/gateway/configuration-examples

最终结论：把配置理解成六层 Agent Runtime 控制模型

- ① Capability
models / plugins / skills / MCP
↓ 系统有什么能力
- ② Identity & Routing
agents / channels / bindings / session
↓ 谁在执行，谁与谁共享上下文
- ③ Context
workspace / bootstrap / startupContext / memory / compaction / pruning
↓ 模型此刻看到什么
- ④ Authorization
tools.profile / allow / deny / sender/provider policy
↓ 模型允许做什么
- ⑤ Isolation
sandbox / workspaceAccess / elevated
↓ 动作在哪里执行，blast radius 多大
- ⑥ Lifecycle & Observability
session / messages / heartbeat / cron / hooks / logging / OTel
↓ Agent 如何长期运行、被监控、被恢复

因此，`openclaw.json` 最值得借鉴的不是某个单独字段，而是它把一个生产级 Agent 系统拆成了可配置、可覆盖、可审计的 Runtime 控制面。真正的工程目标是让每次 Agent turn 都在明确的 Capability、Identity、Context、Authorization、Isolation 与 Lifecycle 策略下发生。